Helping Eyes



Neeraj P Rao (1RVU23CSE303) · Parveen (1RVU23CSE330) · Misha (1RVU23CSE269)

CS4231 · COMPUTER VISION · NOVEMBER 2026

Guide: Prof. Ashwini Kumar Mathur · RV University

Printed text is still a barrier

Existing reading aids fall short in five ways

Cost

Most commercial reading aids are expensive.

Dedicated hardware

They need special devices, not the phone or laptop people already own.

Low light

Real-time reading in poor light is hard for traditional OCR.

Accuracy

Blur, busy backgrounds and varied fonts break simple OCR pipelines.

Information overload

Reading a whole label aloud means listening to everything to find one detail.

Ask, don't listen to everything

Capture once, then ask for exactly the part you need

The camera finds the text by itself, coaches you until it is in view, and captures it. Then you speak or type a request and hear a short answer taken only from that text.

Pages · books · medicine packs
food labels · signs · screens

“Read only the dosage”
Reads just the requested part

“When does it expire?”
Date compared with today in code, then spoken

“Is this safe for children?”
Answered from the printed text, or “the text doesn't say”

“Read everything”
Full text exactly as recognised, no AI model needed

“Look it up online”
Web search only after you agree

Objectives

01

Detect text automatically

From a live camera feed, on any object

02

Read accurately and fast

With Apple Vision, built into macOS

03

Ask for what you need

Instead of listening to the whole label

04

Never invent details

Answer only from the captured text

05

Read books hands-free

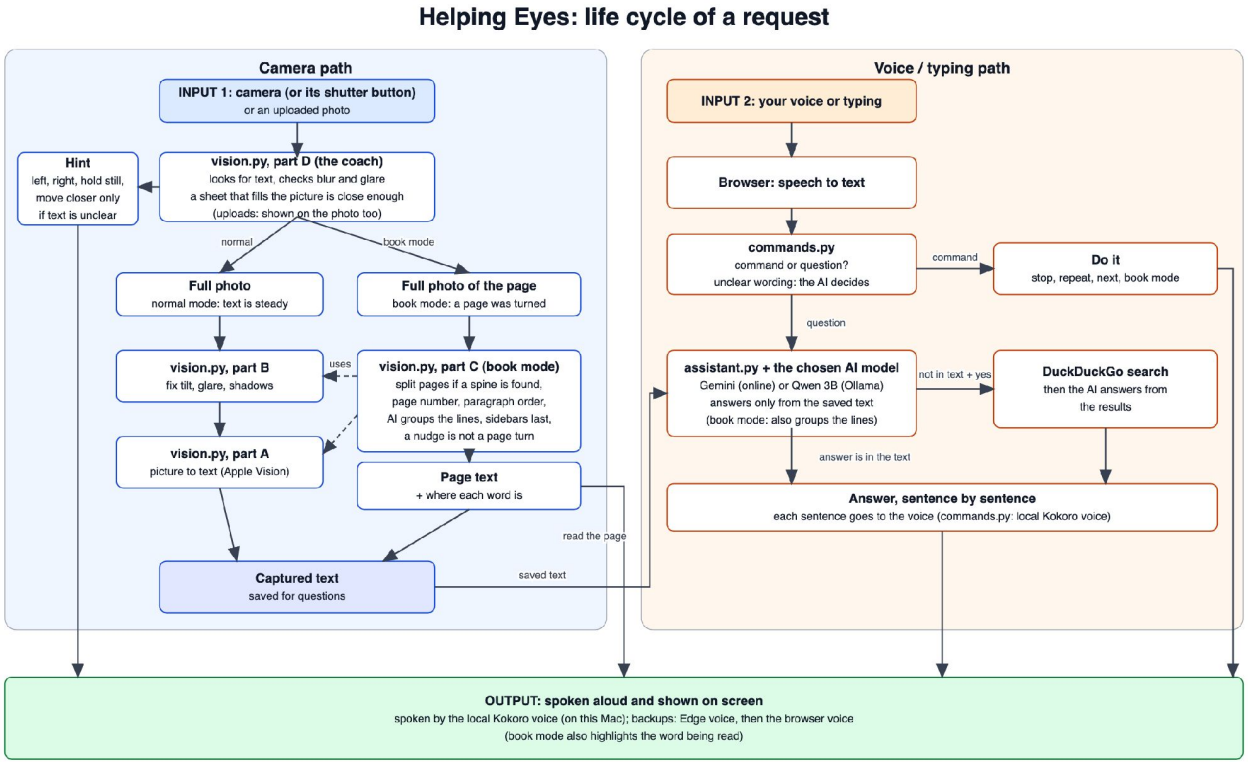
Page by page, no buttons between pages

06

Ordinary hardware

A Mac, any browser, a free model key: no separate GPU, nothing paid

Architecture / Life cycle of one request



An image of a label is hard to read in six ways

Blur

A hand-held camera shakes; soft edges turn letters into smudges.

Glare and dim light

Foil and gloss bounce light into the lens; shadows hide print.

Tilt and perspective

Shot at an angle, a rectangle becomes a slanted four-sided shape.

Curved surfaces

Book pages bend into the spine; bottles wrap around.

Busy backgrounds

Hands, shelves, other packs and patterned tables surround the item.

Books

Two pages, columns, headers, page turns: what order?

The extra problem: the user can’t see the viewfinder, so the camera must find the text itself and talk them into position.

So the camera has six jobs to do before anything is read aloud

01 Spot text

Notice text in the live camera, on any object

02 Judge the photo

Is it sharp, bright enough, free of glare?

03 Coach and capture

Speak directions, then take the photo by itself

04 Fix the photo

Flatten tilt, straighten curves, remove shadows

05 Read it

Get every line and word, with its position

06 Read books

Page turns, two pages, reading order, hands-free

All of it runs in real time on an ordinary Mac: no GPU and no models trained by us.

Three tools clean up the picture

Tool	How it works	Why we use it
Gaussian blur	Each pixel is replaced by an average of itself and its neighbours; nearer neighbours count more. A single odd pixel gets pulled towards the pixels around it.	Camera noise makes single pixels flicker. Smoothing first means later tests react to real shapes (letters, edges), not to random specks.
Thresholding (0 or 255)	Pick a cut-off brightness: darker pixels become black, brighter ones white. **Otsu's method** picks the one cut that best splits the photo into two groups; **adaptive** picks a separate cut for every small area.	It turns "is this ink or paper?" into a clear yes or no. Adaptive copes with shadows, where one cut for the whole photo would fail.
Morphology (open, close)	Works on a black-and-white mask. **Erode, dilate, Open, Close**	Cleans masks: opening removes white specks that are not glare; closing fills the dark print holes inside a page so the page becomes one solid shape.

Three tools measure what is in the picture

Tool	How it works	Why we use it
Contours and components (Glare detection)	Touching white pixels are grouped into separate blobs. Each blob is counted, measured (its size and centre) and its outline is traced.	To find the biggest glare patch and where it is, and to get the outline of a page or a pack.
Projection profile	Add up the ink in every row to get one number per row. Text lines show up as peaks, the gaps between them as valleys. (slide 25)	One cheap curve tells us line spacing, how lines bend on a curved page, the slant, the blank gap at a book's spine, and where columns split.
Convolution	Slide a small grid of weights over the picture. At each spot, multiply each pixel by its weight and add the results.	Used to extract features from images, such as edges, textures, patterns, and shapes, or to apply effects like blurring and sharpening.

PART A AND PART B

Text Recognition and Image Quality Assessment

Before reading, the app asks one question: is this photo good enough? If not, it tells the user what to fix.

Apple Vision reads the letters

What the OCR returns

- The text and a confidence from 0 to 1
- A slanted box: four corners that follow the line
- A box for every word, used for the reading highlight

About 0.1 s a frame on the Mac.

Quick check on live frames

8 or more letters or digits counts as text; One box groups all detected text lines into a single region for the coach.

Careful read on the capture

Read several cleaned-up versions; keep the one the OCR trusts most:

```
score = Σ confidence × characters
```

Tried in order: original, flattened (if tilted), tone-fixed and black-and-white (if quality is poor). The original gets a 5 % head start.

Blur check: sharp print has sudden edges, blur smooths them out

Step 1: compare each pixel with its four neighbours (up, down, left, right)

On plain paper (all about 230)

`230 + 230 + 230 + 230 - 4 × 230 = 0`

Nothing changes, so the answer is 0.

On the edge of a letter (ink is 30)

`230 + 230 + 30 + 30 - 4 × 30 = 400`

A sudden change, so the answer is big.

Step 2: do it everywhere and measure the spread (variance)

- Sharp photo: crisp edges give many big answers next to many zeros, so the spread is large
- Blurry photo: edges are smudged, every answer stays small, so the spread is small

Step 3: spread below 60 → “Hold still, the image is blurry”

Light check: three simple numbers about brightness

Number	How it is worked out	Problem when
Average brightness	Add up every pixel's grey value (0 = black, 255 = white) and divide by the number of pixels	Below 50: too dark → "Low light"
Share of bright pixels	The fraction of pixels brighter than 245	Average above 205 and over 40 % bright → "Too bright"
Contrast	How far pixels spread from the average (standard deviation)	High contrast: Text and objects stand out clearly from the background. Low contrast: Text and objects blend into the background. (>20)

Only one hint is spoken at a time, most urgent first:

1 Low light | **2 Glare** | **3 Blurry** | **4 Too bright**

Glare check: look for a solid patch of pure white

1

Mark pure white

Pixels at 250 or more (out of 255) are marked white; everything else black.

2

Remove specks

“Opening” shaves every white area thin, then regrows it: specks vanish, solid patches stay.

3

Find the biggest patch

Group touching white pixels into blobs; take the largest and its centre.

4

Decide and speak

Covering 1 % of the picture or more means glare; its centre gives the side.

The bright glare is located on the right side of the image, so the user is asked to tilt the page.

Glare hides the print under it completely, so the user has to tilt the page.

PART B

Fixing the photo before reading it

Find the item, flatten it if it is tilted, straighten curved lines and remove shadows.

GrabCut cuts an object out of its background using colour

1

You mark what you know

Some pixels are surely object, some surely background, the rest unknown.

2

It learns the colours

It builds a colour model for the object and one for the background.

3

It sorts the unknowns

Each unknown pixel joins the side it matches; neighbours stick together.

4

It repeats

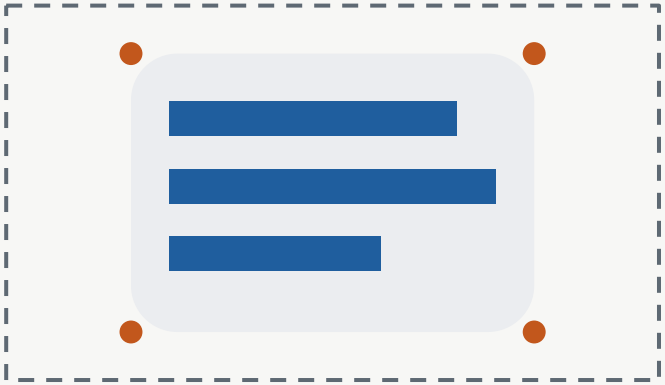
Colour models improve each round; the edge settles in a few rounds.

Like a smart “magic wand”: it cuts out an object by colour, not brightness.

A standard OpenCV function (cv2.grabCut). Next: how we use it to find a pack.

We give GrabCut the text as a starting point to find the item

1. Sure object: the text boxes the OCR found (text is always printed on the item)
2. Maybe object: the area around the text, up to 5 times the text's size
3. Sure background: the picture's border, unless it has the item's colour
4. GrabCut sorts the rest by colour; we keep the piece holding the text
5. Simplify its outline to 4 corners; if it isn't a clean four-sided shape, give no outline



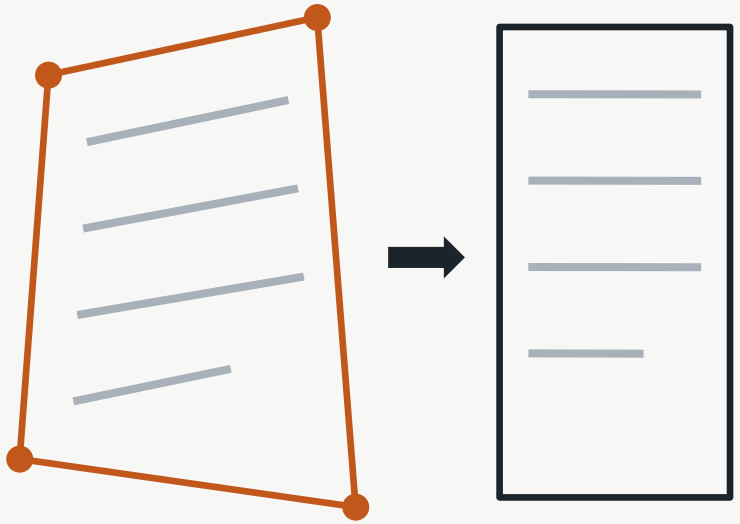
Blue: sure item. Dashed: maybe

Why colour? Picking "the bright part" found the floor: 144 of 145 outlines were the photo's own frame. With GrabCut, outlines were found on 150 of 164 photos.

Flattening a tilted page: move its four corners back to a rectangle

A poster photographed from the side looks like a slanted shape. We stretch it back into a rectangle.

1. Take the item’s 4 corners (previous slide)
2. Measure the size from the longest edges
3. Work out the stretching rule from the 4 corner pairs (perspective transformation)
4. Move every pixel with that rule
5. Skip it if the page is already straight (under 2°)

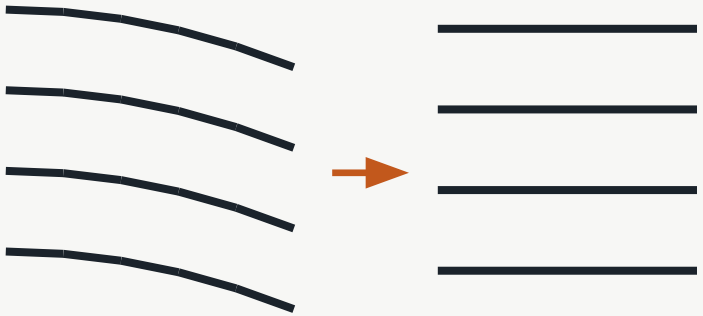


Before: tilted

After: flat

Curved book lines are straightened column by column

1. Divide the page into 16 vertical sections.
2. Count the dark pixels in each row. Text lines appear as peaks because they contain more ink.
3. Compare neighbouring sections to find how much the text lines have shifted up or down.
4. Draw a smooth curve using those shifts to understand how the page is bent.
5. Apply the correction only if the text becomes at least 8% sharper.



Lines bending into the spine, straightened by moving each column up.

```
How to move the pixels in order to make it straight:
dst(x, y) = src(x, y + shift(x))
```

Slant and shadows are corrected only when the checks call for it

Slant

- Rotate the ink mask from −25° to +25° in 1° steps, then 0.1° around the best
- Score each angle by how sharply the rows jump between ink and paper
- Fix only slants of 1.5° or more that score 15 % better than level
- Rotate on a larger canvas so no corner is cut off

Shadows and contrast

- Estimate the paper’s brightness: shrink, dilate to erase the print, blur heavily
- Divide it out, which removes shadows
- CLAHE improves contrast in different parts of an image by dividing the image into grids.
- CLAHE evens out contrast in 8×8 tiles (clip limit 2.0)
- Last resort: adaptive threshold to pure black and white
- Geometry is unchanged, so OCR boxes stay valid

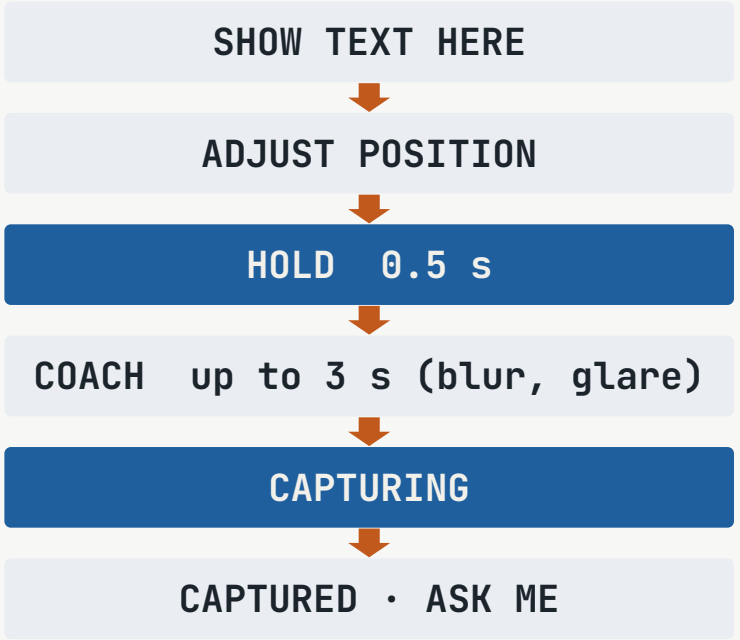
PART D

Coaching someone who can’t see the screen

The checks become spoken directions, and the camera takes the photo by itself.

The camera coaches the user until the text is steady, then captures

- Ready when 65 % or more of the text box is inside the guide box
- “Move closer” when the text is under 14 px tall (at 720p), the OCR is unsure (below 0.8) and the item fills less than 75 % of the view
- “Move left / right / up / down” when the text touches the box edge and its centre is over 80 px off
- Kept quiet: the same hint repeats after 2 s, a quality hint every 5 s, “Low light” every 30 s



PART C

Book mode: reading a book hands-free

Three questions: when to read, where the pages are, and in what order to read them.

Book mode reads a page once the view has been still for 0.8 s

Is anything moving?

- Motion = mean |frame − previous frame| on a blurred 320×180 grey copy
- The median of the last 5 frames, so one noisy frame can’t reset the timer
- Noise floor learned from still frames: noise ← 0.95·noise + 0.05·motion



Moving above noise + 3; still below noise + 1.5.

New page or same page?

An 80×45px black-and-white fingerprint of the layout, each text line smeared into a band, compared at up to 4 cells of shift.

0.01	same page, shifted
0.28	different page
0.06	threshold

The spine is found from its shadow or from the blank strip between pages

1. Find the book's border: Apple's document detector (corners 60° to 120°), or GrabCut from both pages' text

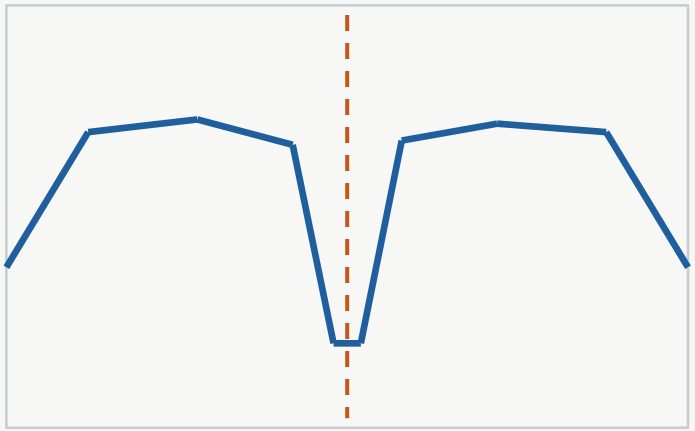
2. A spread only if the book is over 1.1× wider than tall

3. Shadow clue: a dark valley in column brightness, at least 12 % darker than both pages

4. Blank-strip clue: text never crosses the spine, so the smoothest strip (local std of brightness) near the middle

5. Check: a real block of text on both sides, otherwise it is a two-column page

Column brightness



The dip in the middle (dashed) is the spine's shadow.

No clue: use the border centre.

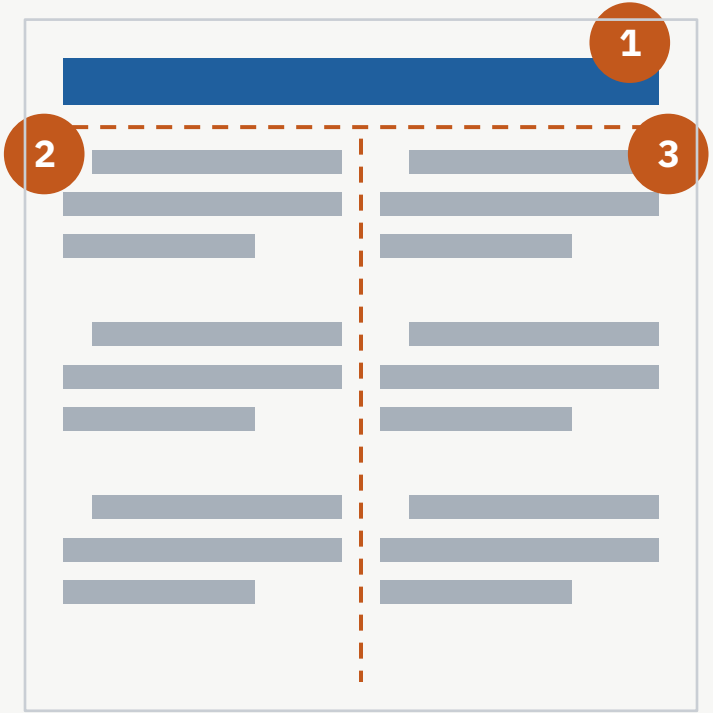
Recursive XY-cut puts columns, headings and paragraphs in reading order

1. Project the line boxes onto the x axis; an empty band wider than 1.2 line heights splits columns, left first

2. Otherwise project onto the y axis; a gap clearly larger than normal line spacing splits heading from body, top first

3. Repeat on each part

4. A part that can’t be cut is read line by line; an indented line starts a new paragraph



Header and footer rows give the printed page number (47, - 47 -, Page 47, xii) and the running title: announced, not read.

The highlight follows each spoken word, and reading resumes after a nudge

Highlighting the spoken word

1. OCR gives a box for every word: The system detects each word and draws an invisible rectangle around it. In the test, all 169 out of 169 words were successfully located.

2. Link every character to its word: Each character in the extracted text is connected to its corresponding line and word position. This helps the system know exactly where each word appears.

3. Highlight the spoken word: The browser estimates which word is currently being spoken using the audio timing and draws an outline around that word on the screen.

4. Map boxes back to the original camera image: Image corrections like rotation, perspective correction (homography), and dewarping change word positions. The system converts the word boxes back to their original camera positions.

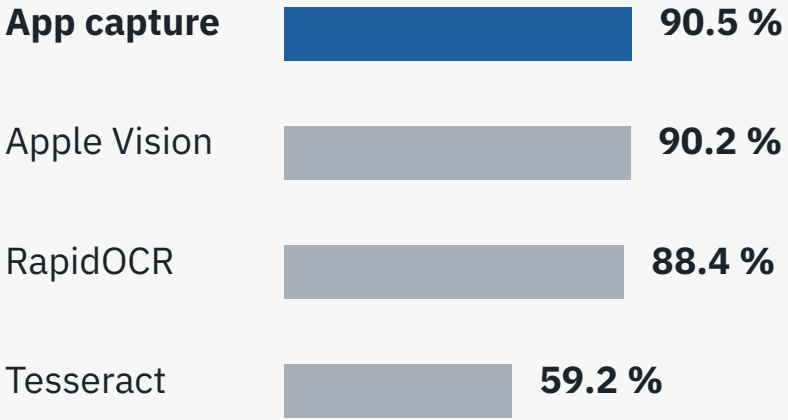
RESULTS

How well it works on real photos

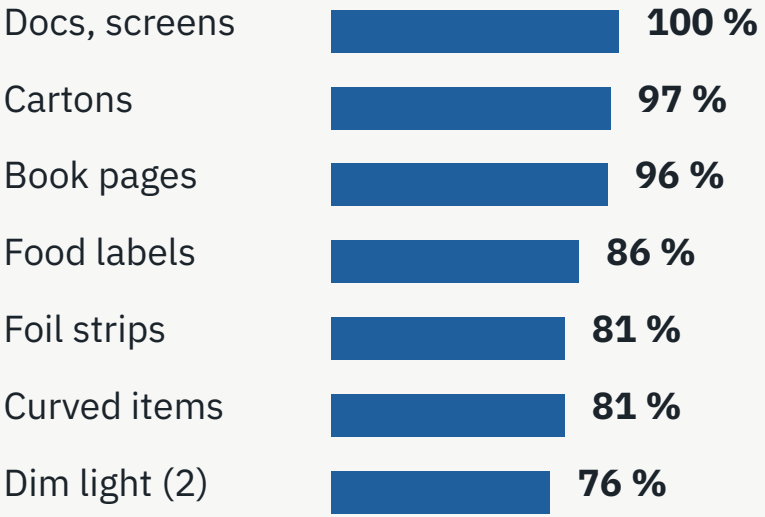
164 phone photos, a real book and a page-turn video, tested on 28 and 29 October 2026.

Apple Vision read 90 % of key lines on real photos; Tesseract read 59 %

By OCR engine (key lines read)



By kind of item (app capture)



App capture = Apple Vision plus our photo fixes (flattening, shadow and contrast fixes): it reads every version and keeps the best. 164 phone photos, M3 Pro, 28 Oct 2026; character error rate 0.095.

On real photos, blur never failed and the glare rule never fired

Check	What we measured	What it means
Blur	Sharpness 248 to 7,934 (median 2,009); never under 60	Hand-held photos were sharp enough
Glare	Never flagged; biggest pure-white blob 0.12 % (rule needs 1 %)	Shine on foil is bright, not pure white: the rule needs a redesign
Light	Brightness 66 to 167; never dark (<50)	Taken in a lit shop; “dark” patches were dark print
Slant	Found on 65 photos, median 3°, up to 26°	Apple Vision copes with moderate slant
Item outline	First rule: 144 of 145 were the photo frame; GrabCut: 150 of 164	Growing from the text fixed it
Best version	Original 143, flattened 16	Flattening helps a minority of tilted packs

Book mode finds the spine and reads pages in the right order

98 %

line pairs in reading order

6 of 6

test spreads split at the spine

4 of 4

page turns caught (+1 early)

Step	Time
Page-turn detector, per frame	0.2 ms
Spine and layout analysis	8 ms
Whole book page, including OCR	0.4 to 0.9 s
Label capture, 1920 px photo to text (typical / slow)	0.29 / 0.47 s

Next: better glare detection and faster live checks

What we found	What we would try next
The glare rule never fired: shine is bright, not pure white	Look for bright, colourless patches on the text (HSV: V > 235 and S < 40)
One page was read while a hand was still turning it	Require the text to be unblocked, or a longer settle time
Apple Vision is macOS-only; foil strips reach 81 %	Try RapidOCR (88.4 %, cross-platform) as a second reader

Thank you

github.com/neerajprao/Helping_Eyes

Try it

`python run.py`
checks the setup, starts the server, opens Chrome

`python run.py --share`
public https link for a phone, kept private by an access code